

Test Starter Guide

A practical way to start testing the Workbench: what to write first, how to cover the API and the UI, how to prove the risk score is right, and how to prove the workflow only ever closes on a value it is allowed to close on.

How to start

Three steps, roughly an hour, and you have a first test running.

1 Get the stack up

Everything below needs a database. Two terminals:

```
bash ops/setup-local-db.sh          # Postgres + Azurite in Docker
dotnet run --project src/1-API/Humaid.RiskGovernance.AdminUI.Web
```

Check it worked: `curl http://localhost:5210/api/Ping` should return `"pong"`.

2 Run what already exists

```
cd tests/e2e && npm install && npm run test:api
```

12 tests, about 3 seconds. This proves your environment is good before you write anything.

3 Pick one user story and cover it end to end

Not one layer across all stories — *one story through all its layers*. You learn the whole shape once, then repeat. Start with **US-7.1 (risk score)**: it is pure logic, needs no AI key, and it is the thing most likely to be asked about in a demo.

Which layer covers what

QUESTION	LAYER	WHY THERE
Is the score arithmetic right?	API + DB	Deterministic — feed inputs, check the number
Can a bad value get in?	DB	The database is what rejects it; only a direct connection sees that
Does the workflow move correctly?	API	State changes happen over HTTP
Can the wrong person do it?	API	Authorization lives at the HTTP boundary
Can a real user complete the job?	UI	Only a browser proves the screens join up

THE ONE RULE THAT KEEPS THIS SIMPLE

Test each thing at the cheapest layer that can actually see it. A browser test that checks a number is the most expensive possible way to check a number. A UI test should only ever ask "can a person get through this?"

1 · API tests — start here

Cheapest, fastest, and where most of the real defects have been.

You already have the harness: `tests/e2e/api/`, Playwright's `request` fixture, no browser involved. A test is just "call an endpoint, check the answer".

The pattern

```
test('US-10.1 — a mitigation factor of 1.0 is rejected', async ({ request }) => {
  const res = await request.post('/api/Scoring/Config', {
    data: { riskCategoryId: PRODUCTS, maxMitigationFactor: 1.0,
      reason: 'probe', actorUserId: analyst.id }
  });
  expect(res.status()).not.toBe(200);
});
```

Name the test after the story it proves. That is what makes traceability free later — the runner writes test names into the results file, so a story with no test shows up as an empty row by itself.

What to cover per endpoint — four questions

#	ASK	EXAMPLE
1	Happy path — does it work with good input?	Submit a valid change request → 200 + a <code>CR-2026-xxxxx</code> number
2	Bad input — is it refused clearly?	Blank override reason → refused, not saved
3	Wrong person — is the role checked?	Product Owner casts a committee vote → should be 403
4	Wrong time — is the sequence enforced?	Vote before routing → refused

Four tests per endpoint, roughly 45 endpoints. You will not do all of them in nine days — and you should not. Do all four for the **write** endpoints that change state (vote, finalize, override, config), and happy-path only for the reads.

EXPECT QUESTION 3 TO FAIL TODAY

No endpoint currently returns 403 for a role violation — the acting user is read from the request body and never checked. When a test fails for this reason, that is **DEF-002**. Mark it expected-to-fail rather than deleting it, so the suite stays green and the test starts shouting the moment somebody fixes it.

2 · Is the risk score appropriate?

The good news: scoring is pure arithmetic with no AI in it, so this is fully provable — not a judgement call.

The formula, and the only four inputs

$$\text{residual} = \text{inherent_rating} - (\text{control_effectiveness} \times \text{max_mitigation_factor})$$

INPUT	ALLOWED RANGE	WHERE IT COMES FROM
inherent_rating	1 – 5	The analyst
control_effectiveness	0 – 1	The analyst
max_mitigation_factor	0 – 0.999... strictly below 1	Configuration · currently seeded at 0.85
residual_rating	> 0 always	Calculated output

Why residual can never be zero — and how to prove it

This is the product's central risk rule: *controls mitigate risk, they never eliminate it*. It holds because of the bounds, not because anyone remembers to check:

$$\begin{aligned} \text{worst case} &= \text{lowest inherent} - (\text{highest effectiveness} \times \text{highest factor}) \\ &= 1 - (1 \times 0.85) \\ &= 0.15 \qquad \leftarrow \text{still above zero} \end{aligned}$$

Since `max_mitigation_factor` is capped *strictly* below 1, and inherent is at least 1, the subtraction can never reach the full inherent value. That is arithmetic, not policy.

Test it as a truth table

A deterministic formula with three inputs is best tested at its **corners** — the boundary values — plus one ordinary case. Nine tests cover it completely.

#	INHERENT	EFFECT.	FACTOR	EXPECTED RESIDUAL	PROVES
S-01	1	1	0.85	0.15	The floor — worst case still above zero
S-02	5	0	0.85	5.00	The ceiling — no controls, no reduction
S-03	3	0.5	0.85	2.575	An ordinary case
S-04	5	1	0.85	4.15	Maximum possible mitigation
S-05	0	—	—	Rejected — inherent below 1	
S-06	6	—	—	Rejected — inherent above 5	
S-07	3	1.5	—	Rejected — effectiveness above 1	
S-08	3	0.5	1.0	Rejected — factor must be < 1 (US-10.1)	
S-09	override residual to 0			Rejected — an analyst cannot zero it either	

Rows S-01 to S-04 go through the API: post a calculation, check the returned number. Rows S-05 to S-09 are **rejections** — the value should never be stored at all — so assert both that the call is refused *and* that nothing landed in the table.

CHECK THE GUARD EXISTS AT BOTH ENDS

The rule is enforced in two places: a C# check in `ScoringService`, and a `CHECK` constraint in PostgreSQL. Test both. Run S-08 through the API (proves the C# guard) and again as a direct `INSERT` against the database (proves the constraint). If only the C# guard exists, any future code path that skips the service bypasses the rule entirely.

What this does *not* prove

These tests prove the arithmetic is right. They do **not** prove the *inputs* were sensible — whether "inherent risk 4" was the correct judgement for this change is an analyst's decision, not a testable fact. That is the boundary between testing and professional judgement, and it is worth saying out loud in a demo.

ONE REAL GAP TO KNOW ABOUT

US-7.1 also requires that you can *trace which controls were credited*. The `control` table is created but never seeded, so the controls list is empty for every category and `control_effectiveness` is just a number someone types in. The arithmetic works; the explainability does not. That is **DEF-005**.

3 · Does the workflow close on an allowed value?

The workflow is a state machine with a fixed, published list of values. That makes it exhaustively testable.

Every allowed value in the system

These are enforced by database `CHECK` constraints, so they are the real contract — not documentation that might be stale:

FIELD	ALLOWED VALUES
<code>change_request.status</code>	Submitted · InAssessment · PendingCommittee · Decisioned
<code>assessment.status</code>	Draft · Finalized
<code>narrative_section.status</code>	AiDrafted · AnalystReviewed · AnalystEdited
<code>committee_vote.vote</code>	Approve · Reject · Defer · ApproveWithConditions
<code>committee_decision.resolution</code>	Approved · Rejected · Deferred · ApprovedWithConditions
<code>change_type</code>	Product · Feature · Process · Vendor · Geography · CustomerSegment
<code>app_user.role</code>	ProductOwner · Analyst · CommitteeMember · Admin
<code>policy_reliance.decision</code>	ReliedUpon · NotRelevant
<code>category_mapping.source</code>	AiProposed · AnalystAdded
<code>extracted_field.source</code>	AiExtracted · AnalystCorrected
<code>risk_score.scored_by</code>	System · Analyst



Figure 1. Two gates matter: finalization controls entry to the committee, and quorum controls the close. Everything else is a value check.

Four things to test — in this order

A • Only allowed values are accepted

For each field above, send a value that is not on the list and confirm it is refused. `vote: "Maybe"`, `changeType: "Partnership"`, `decision: "Unsure"`. This is quick to write and catches a whole class of problem in one pass.

B • The order is enforced

TRY THIS OUT OF ORDER	EXPECTED
Vote before the assessment is routed	Refused — ALREADY WORKS
Finalize while a narrative is still <code>AiDrafted</code>	Refused, and it names what is outstanding
Finalize with no policy decision on a mapped category	Refused
Finalize with no categories mapped at all	Should be refused — CURRENTLY ALLOWED, DEF-004

C • The close resolves to the right value

This is the one to get right, because the rule is deliberately conservative rather than a simple majority. At quorum (currently 2 votes), the resolution is decided in strict priority order:

```
any Reject          → Rejected
else any Defer       → Deferred
else any ApproveWithConditions → ApprovedWithConditions
else (all Approve)   → Approved
```

#	VOTES CAST	RESOLUTION MUST BE
W-01	Approve + Approve	Approved
W-02	Approve + Reject	Rejected — one reject wins, it is never outvoted
W-03	Approve + Defer	Deferred
W-04	Approve + ApproveWithConditions	ApprovedWithConditions
W-05	Reject + Defer	Rejected — reject outranks defer
W-06	ApproveWithConditions + Defer	Deferred
W-07	One vote only	No resolution yet — below quorum
W-08	Two ApproveWithConditions	Both members' conditions text preserved, neither dropped

These eight are already covered by the developers' unit tests and pass. Worth re-running at API level once, because unit tests mock the database — they prove the *rule*, not that the resolution is actually written to the record the Product Owner sees.

D · The record is complete when it closes

When status reaches `Decisined`, check: every voter is named individually, each rationale is stored, conditions text survives, and the audit trail contains the whole chain. *"Decisined"* with an incomplete record is worse than not closing at all.

4 · UI tests — last, and fewest

Three journeys, not thirty. Here is why, and what they should check.

UI tests need the Vite dev server plus both .NET services plus Postgres. They are the slowest and most fragile tests you will own, so they should answer only the question no other layer can: **can a real person actually complete their job?**

The three journeys — one per role

JOURNEY	STEPS	CHECK
Product Owner	Pick change type → fill the form → submit → open My Requests	A <code>CR-</code> number appears; the request is listed with its status
Analyst	Open the inbox → open the workspace → walk the seven tabs → finalize → route	All stages reachable; the audit tab already names the submitter
Committee	Open the queue → review → vote Approve	The vote is recorded <i>against that member by name</i>

Run them **serially, as one story** — the same request moving through the workflow, not three unrelated fixtures. That is what the product actually does, and it is the demo.

Signing in needs no automation

While Auth0 is unconfigured, the app reads the acting user from `localStorage.devUserId`. Setting that key before the page loads *is* the sign-in — no login form to drive. Usually the most painful part of UI testing; here it is one line.

Three selector traps, already hit

TRAP	FIX
Sidebar "Submit Request" vs form "Submit request"	Use <code>exact: true</code> — matching is case-insensitive without it, and you get a strict-mode error
Change type is not a <code><select></code>	It is a combobox; scope to the form — the header has another one
Workspace stages are not buttons	They are <code>role="tab"</code>

What UI tests should never do

- **Check a calculated number.** The API proves the arithmetic in milliseconds.
- **Check a status code.** A browser is the most expensive possible way.
- **Test every field's validation.** One example in the UI to prove validation is wired; the rest at API level.

THE GROWTH RULE

Before adding a UI test, ask: *could the API layer prove this?* If yes, put it there. Twelve UI tests is a ceiling, not a target.

The 5-day plan

Ordered so each day's work makes the next day's easier.

DAY	DO THIS	YOU END WITH
1	Stack up; run the existing 12 API tests; write the nine scoring tests (S-01...09)	The risk calculation is provably correct
2	Workflow value tests (A) and ordering tests (B)	No invalid value or out-of-order transition can slip through
3	Resolution tests (C) and record-completeness (D)	The workflow closes correctly on all four outcomes
4	Role tests — question 3 on every write endpoint	The authorization gap is measured, not assumed
5	The three UI journeys; wire the suite into the PR pipeline	The golden path is protected on every change

A note on what you will find

Several of these tests will fail on day one. That is the point — a test that cannot fail is not a test. When one fails, decide which it is:

IF...	THEN
The product is wrong	Raise a defect; keep the test asserting the <i>correct</i> behaviour, marked expected-to-fail so the fix announces itself
The test is wrong	Fix the test
The requirement was never clear	That is a BA finding — write the missing acceptance criterion

The third case is the most valuable and the easiest to miss. DEF-004 started exactly there: US-6.3 never said what should happen when an assessment has no categories, so the code treated silence as permission.

KNOWN BEFORE YOU START — SO A FAILURE IS NOT A SURPRISE

Role tests will fail (DEF-002 — no endpoint checks role). **Finalize-with-no-categories will pass when it should fail** (DEF-004). **The controls list is empty** (DEF-005). **The first workspace open returns 500** (DEF-006). **Unknown ids return 500 instead of 404** (DEF-003). All five are raised, reproducible and owned — you are confirming known defects, not discovering the product is broken.

Allowed values and numeric bounds in this guide were read directly from the database `CHECK` constraints in `src/4-Persistence/...DB/schema/`, not from documentation — so they are the contract the system actually enforces. Companion documents: *UI, API and Data Test Strategy* (the full version), *QA Test Case Register*, *Test Strategy Alignment Review*.